

Course Outline: Serializers

Title: Serializers: Safe Data Transfer in FastAPI **Skill:** Skill 36: Optimizar la arquitectura para alto tráfico empleando Serializers **Learnpack:** + Serializers **File:** s36-w13d37-serializers **Prerequisite:** Skill 33 — Relacionando objetos y tablas utilizando un ORM **Target Audience:** Beginner developers building FastAPI backends with SQLAlchemy **Total Lessons:** 13 **Format:** Mixed (23% hands-on — 3 lessons)

Course Description

Your FastAPI backend talks to a database. Your database has models. But the shape of data that lives in your database is not the shape of data that should leave your API. Serializers are the layer that enforces that boundary — controlling what gets in, what goes out, and making sure neither direction causes a security incident.

In this course, students learn what a serializer is, how Pydantic implements serializers in FastAPI, and why the boundary between database models and API responses is one of the most important architectural decisions in a backend system.

Course Philosophy

This course emphasizes:

- The boundary between internal data (database models) and external data (API responses) is sacred — always enforce it explicitly
 - Pydantic makes serialization ergonomic, but understanding *why* you need it matters more than learning the syntax
 - Small, focused schemas are always better than large, generic ones — expose only what the current screen actually needs
-

Course Statistics Summary

Section	Lessons
00 - Welcome	1
01 - Serializer Fundamentals	3
02 - Input and Output Handling	3
03 - Safe Data Transfer	4
04 - Assessment	1
05 - Outro	1
Total	13

00.0 What Are Serializers and Why They Matter 📖

Learning Objectives:

- Recognize the problem that serializers solve in a real API
- Understand the role a serializer plays between a database and a client
- Know what this course covers and how its sections connect

Content Outline:

- The leaking endpoint: a FastAPI route that returns a raw SQLAlchemy User object — including `password_hash`, `internal_flags`, and `created_by_admin_id`
- Why this happens: SQLAlchemy models represent database rows, not API contracts
- What a serializer does: it defines the exact shape of data that crosses a system boundary
- The two directions: inbound (validating what arrives) and outbound (controlling what leaves)
- Where Pydantic fits: FastAPI's built-in serialization layer

Transitions: This lesson frames the problem. Section 01 explains what serializers are and how to build them. Section 02 covers input validation and large outputs. Section 03 tackles the most important rule in this course: never expose your database model directly.

Key Principles:

- A database model represents storage. A serializer represents an API contract. They are different things.
- You cannot un-send data. Once a response includes a field it should not have, the damage is done.
- Serializers are not optional glue — they are where security, validation, and contract design live.

Examples:

- A `/users/me` endpoint that accidentally returns `password_hash` alongside `id` and `email`
 - A `/chat` endpoint that accepts a `prompt` with no length limit — making prompt injection trivial
 - A `/articles` endpoint that returns `internal_notes` to anonymous users
-

01.0 Serializer Fundamentals 📖

Learning Objectives:

- Define what a serializer is and what problem it solves
- Explain what Pydantic is and how it implements serializers in FastAPI
- Identify the three jobs a serializer performs: shape, validate, and convert

Content Outline:

- What is a serializer? A serializer transforms data from one representation to another while enforcing a defined shape
- The three jobs of a serializer:
 1. **Shape** — define exactly which fields are included and what their types are

2. **Validate** — reject data that doesn't match the expected shape

3. **Convert** — translate between Python objects, dictionaries, and JSON

- Pydantic as FastAPI's serializer: `BaseModel` is both a schema definition and a runtime validator
- Defining a schema: `class UserResponse(BaseModel): id: int; username: str; email: str`
- `model_config = {"from_attributes": True}` — how Pydantic reads SQLAlchemy model instances
- Using a schema as a FastAPI response model: `response_model=UserResponse`
- Section preview: Lesson 01.1 covers data formats (JSON vs multipart). Lesson 01.2 puts it into practice.

Transitions: This is the foundation. Every other concept in this course builds on the idea that a serializer defines a contract. The next lesson covers the two data formats that serializers typically work with.

Key Principles:

- A Pydantic model is not a database model. Both are Python classes, but they serve different purposes.
- FastAPI automatically uses the `response_model` to filter the output — fields not in the schema are dropped silently.
- Pydantic v2 uses `model_validate(obj)` to convert an ORM instance into a schema instance.

Examples:

```
from pydantic import BaseModel

class UserResponse(BaseModel):
    id: int
    username: str
    email: str

    model_config = {"from_attributes": True}

# Usage in a FastAPI route:
# @app.get("/users/me", response_model=UserResponse)
# def get_me(user: User = Depends(get_current_user)):
#     return user # FastAPI applies UserResponse automatically
```

01.1 Data Formats

Learning Objectives:

- Distinguish between JSON and multipart/form-data as data transport formats
- Know when each format is appropriate for a FastAPI endpoint
- Understand how Pydantic and FastAPI handle each format natively

Content Outline:

- JSON: the default format for API responses and structured request bodies
 - All FastAPI `BaseModel` request bodies are JSON by default
 - `Content-Type: application/json`
 - Good for: structured data, nested objects, most API interactions
- Multipart/form-data: for file uploads and mixed text+file requests
 - `Content-Type: multipart/form-data`
 - In FastAPI: use `File(...)` and `UploadFile` alongside `Form(...)` fields
 - Good for: uploading images, documents, audio
- Why you cannot mix `BaseModel` body with `File` in the same endpoint — use `Form` fields instead
- Content negotiation: the client's `Accept` header tells the server which format it expects

Transitions: Understanding formats matters because a serializer's output format affects how the client reads it. Now that we know what serializers are and what formats they use, lesson 01.2 puts it into practice.

Key Principles:

- JSON is for structured data exchanges. Multipart is for binary/mixed content.
- FastAPI infers the request body format from the parameter types — no manual parsing needed.
- When in doubt, use JSON. Only switch to multipart when files are involved.

Examples:

```
# JSON body (default)
class CreatePostRequest(BaseModel):
    title: str
    content: str

@app.post("/posts")
def create_post(body: CreatePostRequest): ...

# Multipart body (file upload)
from fastapi import File, Form, UploadFile

@app.post("/upload")
def upload(file: UploadFile = File(...), description: str = Form(...)):
    ...
```

01.2 Build Your First Pydantic Serializer 🖥️

Challenge Prompt: A FastAPI app has a `User` SQLAlchemy model with fields `id`, `username`, `email`, and `password_hash`. Create a `UserResponse` Pydantic schema that exposes only the safe fields and add `model_config` so it can read from the ORM instance directly.

Solution Code:

```
from pydantic import BaseModel

class UserResponse(BaseModel):
    id: int
    username: str
    email: str

    model_config = {"from_attributes": True}
```

Challenge Code:

```
from pydantic import BaseModel

# TODO: Create a UserResponse schema.
# Include only: id (int), username (str), email (str)
# Do NOT include: password_hash
# Add model_config so it can be built from a SQLAlchemy object.

class UserResponse(BaseModel):
    pass # your code here
```

02.0 Input and Output Handling

Learning Objectives:

- Distinguish between inbound serialization (validation) and outbound serialization (response shaping)
- Understand why both directions require explicit schema definitions
- Know the two sub-topics this section covers: input validation and large output handling

Content Outline:

- Data flows in two directions through every API endpoint:
 - **Inbound:** a client sends data (request body, query parameters) — must be validated before use
 - **Outbound:** the server sends data back — must be filtered before it leaves
- Inbound: Pydantic validates and coerces incoming data automatically when used as a request body
- Outbound: response schemas filter out internal fields and enforce types
- The new challenge in AI systems: outbound data is sometimes not a single JSON object — it's a stream
- Section preview: Lesson 02.1 covers large outputs (streaming and chunking). The challenge in 02.2 tests both directions.

Transitions: Section 01 built the foundation — what a serializer is and how to define one. This section zooms in on the two distinct jobs: controlling what enters and what leaves, including the streaming

case.

Key Principles:

- Input serializers protect your system from bad data. Output serializers protect your users from leaked data.
- Validation happens at the boundary. If you validate only at the database layer, bad data has already traveled too far.
- AI endpoints are different: responses can be large, slow, or continuous — serializers must account for that.

Examples:

- An inbound `ChatRequest` schema rejects prompts over 500 characters before any AI call is made
 - An outbound `ChatResponse` streams tokens one by one instead of waiting for the full response
 - A query parameter validator rejects negative page numbers before any database query runs
-

02.1 Large Outputs

Learning Objectives:

- Explain why large AI outputs require streaming instead of buffered responses
- Describe how chunking breaks a large response into manageable pieces
- Implement a basic streaming FastAPI endpoint using `StreamingResponse`

Content Outline:

- The buffered response problem: when an AI model generates 2,000 words, waiting for the full output before sending anything creates a bad user experience and risks timeouts
- Streaming: sending data incrementally as it becomes available
 - `StreamingResponse` in FastAPI with a generator function
 - `yield` — the Python mechanism that makes a function a generator
 - `media_type="text/event-stream"` for Server-Sent Events (SSE), the format used by ChatGPT-style UIs
- Chunking: breaking a large payload into fixed-size pieces
 - Useful when the full response is pre-computed but too large to send at once
 - Common in file downloads and bulk data exports
- When to stream vs when to chunk:
 - Stream: when output is generated progressively (LLM completions, live logs)
 - Chunk: when output is large but fully available (file downloads, large report exports)

Transitions: Now that we understand both inbound and outbound concerns, the challenge in 02.2 gives you practice applying input validation constraints and examining how streaming works.

Key Principles:

- Never buffer what you can stream. Users experience responsiveness before completeness.
- A streaming endpoint does not return a `BaseModel` response — it returns a `StreamingResponse` with a generator.

- Server-Sent Events (SSE) is the standard format for streaming text in real-time AI UIs.

Examples:

```
from fastapi import FastAPI
from fastapi.responses import StreamingResponse

app = FastAPI()

def generate_tokens():
    tokens = ["Hello", ", ", "world", "!"]
    for token in tokens:
        yield token

@app.get("/stream")
def stream_response():
    return StreamingResponse(generate_tokens(), media_type="text/plain")
```

02.2 Validate and Stream

Challenge Prompt: Complete the `ChatRequest` Pydantic schema by adding Field constraints so that `prompt` is between 1 and 500 characters and `temperature` is a float between 0.0 and 2.0 with a default of 0.7.

Solution Code:

```
from pydantic import BaseModel, Field

class ChatRequest(BaseModel):
    prompt: str = Field(min_length=1, max_length=500)
    temperature: float = Field(default=0.7, ge=0.0, le=2.0)
```

Challenge Code:

```
from pydantic import BaseModel, Field

class ChatRequest(BaseModel):
    # TODO: Add a 'prompt' field (str) with:
    #   - minimum length of 1 (cannot be empty)
    #   - maximum length of 500 characters
    # TODO: Add a 'temperature' field (float) with:
    #   - default value of 0.7
    #   - must be between 0.0 and 2.0 (inclusive)
    pass # your code here
```

03.0 Safe Data Transfer

Learning Objectives:

- Explain the boundary between database models and API responses and why it must be enforced
- Identify what a Data Transfer Object (DTO) is and what problem it solves
- Recognize that security in deserialization is as important as security in authentication

Content Outline:

- The boundary: your database schema is an internal implementation detail; your API response is a public contract
- These two things must never be the same object
- What is a DTO (Data Transfer Object)?
 - A simple object whose only job is to carry data across a boundary
 - In FastAPI: a Pydantic `BaseModel` that defines exactly what the API exposes
 - Named for what it does, not what the data represents internally
- Three problems this section covers:
 1. (Lesson 03.1) Accidentally returning your entire ORM model — exposing fields the client should never see
 2. (Lesson 03.2) Security holes in deserialization — what happens when you trust incoming data too much
 3. (Lesson 03.3) Building a DTO layer that separates these concerns cleanly

Transitions: Sections 01 and 02 established what serializers do and how they handle data flow. This section establishes the most important rule in the course — why your database model and your API response model must always be different objects — and what goes wrong when they aren't.

Key Principles:

- A DTO is a promise to your clients about what they will receive. It is independent of how you store data internally.
- The boundary between database and API is where most data security incidents begin.
- "Safe" data transfer means: the right fields, in the right shape, to the right recipient.

Examples:

- A `UserDB` SQLAlchemy model with 12 fields; a `UserResponse` DTO with 3 fields — that gap is intentional
- A `CreateUserRequest` DTO that accepts only `username`, `email`, and `password` — rejecting any attempt to set `is_admin` via the request body
- An `ArticleResponse` DTO that omits `internal_notes` and `is_draft` from the public response

03.1 Never Expose Your Database Model Directly

Learning Objectives:

- Identify what fields a typical SQLAlchemy model exposes that should never reach a client
- Implement the DTO pattern to decouple database structure from API responses

- Recognize the mass assignment vulnerability and how DTOs prevent it

Content Outline:

- What a SQLAlchemy model actually looks like: it often includes `password_hash`, `created_at`, `updated_at`, `deleted_at`, `is_admin`, `internal_flags` — none of which belong in a public response
- The naive mistake: returning the ORM instance directly without a response schema
 - SQLAlchemy models are not JSON-serializable by default — FastAPI will attempt to convert them, sometimes partially, sometimes with unexpected fields included
 - Using `response_model=UserResponse` prevents this even if the route returns a full ORM object
- The DTO pattern in practice:
 - One Pydantic model per "view" of the data (public response, admin response, internal transfer)
 - Names: `UserPublicResponse`, `UserAdminResponse`, `UserCreateRequest`
 - Each schema includes only the fields appropriate for its audience
- Mass assignment vulnerability: without input DTOs, a client could send `{"username": "alice", "is_admin": true}` and your backend might accept it
 - DTOs prevent this by only declaring the fields you intend to accept
 - Anti-pattern: using `**request.dict()` to pass all incoming fields directly to an ORM constructor

Transitions: DTOs control the shape of data. But serialization also has a security dimension in the *deserialization* direction — when you parse incoming data. That's lesson 03.2.

Key Principles:

- One schema per API contract. Never reuse an ORM model as an API response.
- The `response_model` parameter in FastAPI is your last line of defense — always set it.
- Input schemas must declare only the fields you intend to accept — nothing more.

Examples:

```
# Anti-pattern: returning ORM model directly
@app.get("/users/{id}")
def get_user(id: int, db: Session = Depends(get_db)):
    return db.query(User).filter(User.id == id).first()
    # FastAPI may include password_hash, is_admin, internal flags

# Correct: explicit DTO
class UserPublicResponse(BaseModel):
    id: int
    username: str
    email: str
    model_config = {"from_attributes": True}

@app.get("/users/{id}", response_model=UserPublicResponse)
def get_user(id: int, db: Session = Depends(get_db)):
```

```
return db.query(User).filter(User.id == id).first()
# Only id, username, email are returned – everything else is filtered
```

03.2 Security in Deserialization

Learning Objectives:

- Identify the security risks that arise when deserializing untrusted input
- Explain what object injection and mass assignment mean in practical terms
- Apply Pydantic constraints and `model_config` settings that defend against these risks

Content Outline:

- Deserialization: the process of converting raw incoming data (JSON, form fields) into a Python object
- Why untrusted input is dangerous: a client can send any JSON they want — without validation, that data reaches your business logic unchecked
- Three deserialization risks:
 1. **Extra fields / mass assignment:** client sends fields you didn't expect — some ORM libraries and dict unpacking will accept them silently
 - Defense: Pydantic's `model_config = {"extra": "forbid"}` rejects any field not declared in the schema
 2. **Type coercion abuse:** Pydantic will try to coerce strings to integers — a client sending `"9999999999999999"` for an `id` field will succeed if you allow it
 - Defense: use `strict=True` mode or explicit `Field` validators with constraints
 3. **Deeply nested or recursive payloads:** crafted to exhaust memory or CPU during parsing (a form of DoS)
 - Defense: enforce `max_length` and `max_items` constraints on all collection fields; don't deserialize arbitrary depth
- The principle: validate at the boundary. The serializer is the first thing that touches incoming data — make it strict.
- Anti-pattern: deserializing a payload into a raw `dict`, then passing it directly to an ORM method or an AI model as a prompt

Transitions: Now that we understand both DTOs (outbound safety) and deserialization security (inbound safety), the challenge in 03.3 combines them: build a DTO layer that does both.

Key Principles:

- Never trust incoming data. A Pydantic schema is not just convenient — it is your validation boundary.
- `extra = "forbid"` is cheap and powerful. It costs nothing and prevents a class of mass assignment bugs.
- Prompt injection starts with unsanitized text reaching an AI model. Your input serializer is the first checkpoint.

Examples:

```
from pydantic import BaseModel, Field

class CreateCommentRequest(BaseModel):
    content: str = Field(min_length=1, max_length=2000)
    post_id: int

    model_config = {"extra": "forbid"}

# Client sends: {"content": "nice post", "post_id": 1, "is_approved": true}
# Pydantic raises ValidationError – "is_approved" is not allowed
```

03.3 Build a Secure DTO Layer 📖

Challenge Prompt: A FastAPI app has an `Article` SQLAlchemy model with fields `id`, `title`, `content`, `created_at`, `author_id`, `internal_notes`, and `is_draft`. Create an `ArticleResponse` Pydantic DTO that exposes only the public fields and configures it to read from ORM instances.

Solution Code:

```
from pydantic import BaseModel
from datetime import datetime

class ArticleResponse(BaseModel):
    id: int
    title: str
    content: str
    created_at: datetime

    model_config = {"from_attributes": True}
```

Challenge Code:

```
from pydantic import BaseModel
from datetime import datetime

# TODO: Create an ArticleResponse DTO.
# Include ONLY these public fields:
#   id (int), title (str), content (str), created_at (datetime)
# Do NOT include: author_id, internal_notes, is_draft
# Add model_config so it reads from a SQLAlchemy object directly.

class ArticleResponse(BaseModel):
    pass # your code here
```

04.0 Serializers in Practice 🧐

Learning Objectives:

- Apply serializer concepts to real API design scenarios
- Identify violations of the DTO pattern and deserialization security rules
- Choose the right Pydantic configuration for a given use case

Question Format: Scenario-based (read the situation, choose the best answer)

Questions:

Q1. A developer writes this FastAPI endpoint:

```
@app.get("/profile", response_model=User)
def get_profile(current_user: User = Depends(get_current_user)):
    return current_user
```

The `User` here is a SQLAlchemy ORM class. What is the most significant problem?

A) The route is missing error handling B) SQLAlchemy `User` is being used as the response schema, which may expose `password_hash` and other internal fields C) `Depends` is not the correct way to inject a user D) The endpoint should use POST instead of GET

Correct: B — Using the ORM model as `response_model` exposes internal fields. A dedicated Pydantic DTO should be used instead.

Q2. A student adds `model_config = {"from_attributes": True}` to a Pydantic schema. What does this do?

A) It makes the schema read from environment variables B) It allows Pydantic to read attributes from ORM instances instead of requiring a dict C) It enables strict mode for all fields D) It makes all fields optional

Correct: B — `from_attributes: True` (formerly `orm_mode = True`) tells Pydantic to access data via attribute access (`.id`, `.username`) rather than key access (`["id"]`, `["username"]`), which is what SQLAlchemy objects use.

Q3. A client sends this JSON to a `CreateUserRequest` endpoint:

```
{"username": "alice", "email": "alice@example.com", "password": "secret",
  "is_admin": true}
```

The `CreateUserRequest` schema declares only `username`, `email`, and `password`. With the default Pydantic v2 configuration, what happens to `is_admin`?

A) It raises a validation error B) It is silently ignored C) It is passed to the ORM model D) It causes a 500 server error

Correct: B — By default, Pydantic v2 ignores extra fields. To reject them, you must set `model_config = {"extra": "forbid"}`.

Q4. Which configuration prevents a client from injecting unexpected fields into a Pydantic request schema?

A) `model_config = {"from_attributes": True}` B) `model_config = {"extra": "forbid"}` C) `model_config = {"strict": True}` D) `model_config = {"populate_by_name": True}`

Correct: B — `extra = "forbid"` causes Pydantic to raise a `ValidationError` if any field not declared in the schema is present in the input.

Q5. A FastAPI AI endpoint accepts a `prompt` field with no length constraint. A user sends a 50,000-character prompt. What is the primary risk?

A) The database will reject the insert B) The response will be too large for the client to parse C) The AI model receives unsanitized, potentially injected content with no size guard, risking prompt injection and resource exhaustion D) FastAPI will automatically truncate it to a safe length

Correct: C — Without a `max_length` constraint on the input schema, oversized or crafted prompts reach the AI model unchecked. This is the serializer's job to prevent.

Q6. A developer needs to return a large AI-generated report as a FastAPI response. The report takes 8 seconds to fully generate. What is the best approach?

A) Use `time.sleep(8)` and then return the complete JSON response B) Return a `StreamingResponse` with a generator that yields tokens as they are produced C) Use a Pydantic schema with `max_length = 100000` D) Cache the report and return it on the second request

Correct: B — `StreamingResponse` with a generator sends data to the client as it becomes available, rather than waiting for the complete output. This eliminates the perceived wait time and avoids timeout issues.

Q7. Which of the following correctly describes the purpose of a DTO (Data Transfer Object)?

A) A database table that holds temporary data during migrations B) A Pydantic model that represents the exact shape of a database row C) An object whose only purpose is to carry data across a system boundary, decoupled from the internal data model D) A FastAPI dependency that transfers session data between requests

Correct: C — A DTO decouples what the API exposes from how data is stored internally. It is not a mirror of the database table — it is an independent contract.

Evaluation Criteria:

- 7/7: Serializer principles are solid — ready to apply them in production code
- 5–6/7: Good understanding with minor gaps — review the lessons where you missed
- 3–4/7: Revisit Section 03 (DTOs and security) before moving on
- Below 3/7: Re-read the full course before proceeding

Score Interpretation: The most common mistakes are Q3 (default Pydantic behavior on extra fields) and Q1 (using ORM model as response schema). If you missed either, pay close attention to `model_config` settings in your next FastAPI project.

05.0 Outro

Content Outline:

What you accomplished: You started this course with an API that leaked internal data and accepted unsanitized input. You now have the tools to fix both. You know what a serializer is, how Pydantic implements it in FastAPI, and — most importantly — why the boundary between your database and your API is one you must always enforce explicitly.

The one rule worth remembering: Never use your ORM model as an API response. Write a DTO. Always.

Connection to the bigger picture: Serializers are one layer of a larger architecture for handling data under load. The next learnpack covers caching — another layer that sits between your database and your clients, this time optimizing for speed rather than safety. The concepts connect: a well-defined DTO makes it straightforward to cache specific views of data without accidentally caching sensitive fields.

Next steps:

- Add `response_model` to every FastAPI route in your current project
- Add `model_config = {"extra": "forbid"}` to every request body schema
- Review any route that returns an ORM object directly and replace it with a DTO
- Explore Pydantic's `field_validator` and `model_validator` decorators for custom validation logic beyond `Field` constraints

Resources:

- Pydantic v2 documentation: <https://docs.pydantic.dev/latest/>
- FastAPI response models guide: <https://fastapi.tiangolo.com/tutorial/response-model/>
- FastAPI request body validation: <https://fastapi.tiangolo.com/tutorial/body/>
- OWASP Mass Assignment Cheat Sheet:
https://cheatsheetseries.owasp.org/cheatsheets/Mass_Assignment_Cheat_Sheet.html

Note: This is conclusion only — no theory, exercises, or assessment.
